

LOC3X1/LOC3X2 Camel Project: Current Architecture and Operations

Alberto Hernandez

November 19, 2025

Abstract

This document reflects the current implementation of the generated **result** folder: a Spring Boot 3.3.x + Apache Camel 4.7.x service with a single aggregated Camel YAML route file, per-operation XML REST controllers, and mediation routes for XSLT transforms. It documents inputs (**Dependencies**, **Dependencies2**), contract assembly, XSLT generation from IBM **.map** files, and how to build and run in isolation on port 8081.

1 Technology Stack

- Java 17+ (runtime tested with Java 25).
- Spring Boot 3.3.4.
- Apache Camel 4.7.0 with YAML DSL.
- Starters: `camel-platform-http-starter`, `camel-http-starter`, `camel-xslt-starter`.

2 Inputs and Generation

- Client inputs are `CHUBB/Dependencies` and `CHUBB/Dependencies2`. Only these trees are scanned.
- Contracts (WSDL/XSD) are copied into `src/main/resources/contracts/selected/` preserving service folders.
- Controllers are generated per operation only when a matching Request/Reply XSD pair exists.
- IBM **.map** files found under `Dependencies/Maps` are converted to XSLT into `src/main/resources/xslt/`.

3 Runtime Architecture

3.1 Aggregated Routes

- Single YAML: `src/main/resources/routes/loc-service.yaml`.
- Inbound: `platform-http:/loc/<Operation>` per flow.
- Transform: `to: xslt:classpath:xslt/<...>.xsl` for request/response mapping.

- Forward: `http://localhost:8081/svc/<Operation>?bridgeEndpoint=true&throwExceptionOnFailure=`
- Global `onException` defined first; logs at ERROR and returns a simple XML error body when needed.

3.2 Controllers

- Endpoints: `POST /svc/<Operation>` (XML in/out).
- Each controller references its operation-specific Request/Reply XSDs from `contracts/selected`.
- For integration runs, controllers return a minimal XML reply; strict validation can be re-enabled.

3.3 Mediation

- Java route builder: `com.example.mediation.MediationRoutes`.
- Always includes `direct:mediation/identity` → `xslt:classpath:xslt/identity.xsl`.
- Registers `direct:mediation/<mapName>` for each converted XSLT.

4 Project Layout (result)

- `src/main/java/com/example/controller`: per-operation XML controllers.
- `src/main/java/com/example/mediation`: mediation routes class.
- `src/main/resources/routes`: `loc-service.yaml`.
- `src/main/resources/xslt`: `identity.xsl` and any generated XSLTs.
- `src/main/resources/contracts/selected`: WSDL/XSD organised per service.
- `samples`: auto-generated minimal request samples per operation.

5 Build and Run

5.1 Build

```
mvn clean package -DskipTests
```

5.2 Run

```
java -jar -Dserver.port=8081 target/loc-service.jar
```

or dev mode:

```
mvn spring-boot:run -Dspring-boot.run.jvmArguments="-Dserver.port=8081"
```

5.3 Smoke Tests

```
curl -sf http://localhost:8081/actuator/health
curl -i -X POST -H "Content-Type: application/xml" \
  --data-binary @samples/<Operation>-request-min.xml \
  http://localhost:8081/loc/<Operation>
```

6 Notes and Troubleshooting

- If maps are not present, mediation still works via `identity.xsl`.
- Controllers are generated only when Request/Reply XSDs exist; absent pairs are skipped.
- Use `jar run` for lean checks; use `spring-boot:run` for live log debugging.